Ders 19 Genişletilmiş Notu: Sıralama Algoritmaları: Bubble Sort İyileştirmesi, Selection Sort ve Insertion Sort

Ders 19 Genişletilmiş Notu: Sıralama Algoritmaları: Bubble Sort İyileştirmesi, Selection Sort ve Insertion Sort

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Algoritma Becerisi ve Sıralamanın Yeri
- 2. Bubble Sort Tekrarı
- 3. Bubble Sort'un Karşılaştırma Sayısı ve Karmaşıklığı
- 4. İyileştirilmiş Bubble Sort
- 5. Selection Sort'un Mantığı
- 6. Selection Sort'un İlkel Versiyonu
- 7. İyileştirilmiş Selection Sort
- 8. Insertion Sort'un Mantığı

1. Algoritma Becerisi ve Sıralamanın Yeri

Hoca derse, algoritma geliştirmenin yolunun ne olduğunu sorarak başladı. Cevabı, hayattaki pek çok şeyde olduğu gibi tekrar etmektir. Tekrar ettikçe beyin o şekilde düşünmeye alışır ve ilk başta çok zor görünen tasarımlar zamanla kolay gelmeye başlar. Herkesin yatkınlığı farklıdır. Kimi hızlı adapte olur, kimi farklı bir düşünce sistemine sahip olduğu için zaman içinde alışır.

Mesleğin çalışma alanları çok çeşitlidir ve her yıl yeni konular ortaya çıkar. Ama temel prensip, bu derste görülen tasarlama yöntemlerine dayanır. Burada kazanılan düşünme biçimi, gerçek hayat problemlerini çözerken de destek olur.

Bugünün konusu, kalınan yerden devam edilen sıralama (sort) algoritmalarıdır. Hocaya göre bu algoritmalar bundan sonraki süreçte çok karşınıza çıkacaktır. Bir işi yaparken, mantıklıysa, veriyi önce sıralayıp sonra işlemi yapmayı tercih edebilirsiniz. Bunun için sıralama algoritmalarını iyi anlamak gerekir.

2. Bubble Sort Tekrarı

2.1 Mantık

Bubble sort ikili karşılaştırma yapar. Yan yana, yani arka arkaya gelen iki eleman karşılaştırılır. Küçükten büyüğe sıralıyorsak ve soldaki eleman sağdakinden büyükse ikisinin yeri değiştirilir.

2.2 Örnek: Birinci ve İkinci Tur

Hoca önce ekrana `5 1 3 2 4 7 8 6` yazdı, sonra daha rahat anlaşılsın diye 5'in yerine 9 koydu. Dizi `9 1 3 2 4 7 8 6` oldu. İç döngünün (inner loop) her adımında 9 bir sağa geçer:

- 9 ile 1 karşılaştırılır, yer değiştirir: `1 9 3 2 4 7 8 6`
- 9 ile 3: `1 3 9 2 4 7 8 6`
- 9 ile 2: `1 3 2 9 4 7 8 6`
- 9 ile 4: `1 3 2 4 9 7 8 6`
- 9 ile 7: `1 3 2 4 7 9 8 6`
- 9 ile 8: `1 3 2 4 7 8 9 6`
- 9 ile 6: `1 3 2 4 7 8 6 9`

Dış döngünün birinci turu (first round) böyle tamamlanır. İkili karşılaştırmayla en büyük değer en sağa kadar itilmiş olur. Ardından iç döngüden çıkılır ve dış döngünün ikinci turuna (second round) geçilir.

İkinci tur `1 3 2 4 7 8 6 9` ile başlar. 1 ile 3 karşılaştırılır, yer değişmez. 3 ile 2 karşılaştırılır, yer değişir. Böyle devam edilir ve tur sonunda dizi `1 2 3 4 7 6 8 9` olur. 8'in geldiği yer, dizinin en büyük ikinci elemanının gerçek yeridir.

2.3 Döngü Sınırları: N−I ve N−1

İkinci turda son karşılaştırma 6 ile 8 arasındadır. 8 ile 9'u karşılaştırmaya gerek yoktur, çünkü ilk turda dizinin en büyük elemanı zaten en sona itilmiştir. Bu yüzden iç döngüdeki karşılaştırma sondan bir öncekine kadar getirilir. Bu sınır her turda bir azalır ve azalma I'ya bağlıdır. Bu nedenle iç döngünün sonu N−1 yerine N−I yazılır. Böylece sonda yer alan, zaten sıralanmış elemanlar gereksiz yere birbiriyle kıyaslanmaz.

Dış döngüde her tur bir elemanı yerine koyar: birinci turda en büyük, ikinci turda en büyük ikinci, üçüncü turda en büyük üçüncü eleman yerleşir. Yedinci turun sonunda elde `1 2 3 4 6 7 8 9` vardır. Kalan tek eleman için bir tur daha dönmeye gerek yoktur. N−1 adımda dizinin N−1 elemanı doğru yerine yerleştirilmiştir. Dış döngünün sonu bu yüzden N−1'dir.

Algoritmanın akışı numaralı adımlarla şöyledir:

1. Başla. N ve diziyi al.
2. `I` değişkenini 1'den N−1'e kadar birer birer artır (dış döngü).
3. Her `I` için `J` değişkenini 1'den N−I'ya kadar birer birer artır (iç döngü).
4. `Dizi[J] > Dizi[J+1]` ise yer değiştir: `tmp = Dizi[J]`, `Dizi[J] = Dizi[J+1]`, `Dizi[J+1] = tmp`.
5. Döngüler bitince dur.

Soru: İçteki for döngüsünden önce bir if açıp kontrol yapsak olur mu? **Cevap:** Buna gerek yok. Yerleri değişecek elemanlar için bu kontrol gereksiz yere yapılmış olur. Değişmeyecek olanlar da zaten içeride if'in false koluna gider ve yoluna devam eder.

Soru: C'de iç döngü sınırını N−I−1 yazdım ve doğru sonuç aldım, sebebi nedir? **Cevap:** Algoritma birden başlatıldı, öğrenci ise C'de sıfırdan başlattı. Bu durumda sınırın öyle olması normaldir.

> **Hocanın vurgusu:** Uç değerlerle ilgili kontrollerinizi her zaman yapın.

Hoca bu arada soruları neden hazır bir çözümden bakmadan, öğrencilerle birlikte çözdüğünü de açıkladı. Doğru seçeneği doğrudan yazarsa fikir yürütmek için alan kalmaz. Bu yüzden zaman zaman gözünden kaçan ufak hatalar olabilir. Fark ederseniz uyarın. Sonradan fark ederseniz analiz yaparak hatayı mutlaka düzeltin.

3. Bubble Sort'un Karşılaştırma Sayısı ve Karmaşıklığı

3.1 En Kötü Durum (Worst Case)

Hoca, en kötü durum senaryosunda kaç karşılaştırma gerektiğini sordu. En kötü durum dizisi olarak `9 8 7 6 4 3 2 1` kullanıldı.

Soru: Karşılaştırma sayısı (N−1) çarpı (N−1)! kadar olabilir mi? **Cevap:** Öyle olsaydı "bilgisayarlar ağlardı". Bu çok fazladır.

Bir başka öğrenci, her durumda (N−1) çarpı (N−I) kez kontrol yapıldığını söyledi ve hoca bunu onayladı. Bu ifade genelleştirilip sonuçta ortaya çıkan üssü küçük terimler atıldığında, en kötü durumda N^2'lik bir işlem karmaşıklığı ortaya çıkar.

Soru: Buradan neden N^2 aldığımızı bir daha açıklayabilir misiniz? **Cevap:** Birinci turda N−1 karşılaştırma yapıldı. Bu tur işi yaklaşık N kez yapılıyor. N çarpı N−1 yaklaşık N^2 eder. Tam olarak yazsak N^2'nin yanında örneğin $-2N$ ve $+3$ gibi terimler de olurdu. Üssü küçük olanlar elenerek ifade edilir.

> **Ders sonrası not:** Hoca sayımı "(N−1) çarpı (N−I)" ve "N çarpı N−1" diye söyledi; bunlar büyük olasılıkla mertebeyi göstermek için söylenmiş yaklaşık ifadelerdir. Doğrusu, I. turda N−I karşılaştırma yapılır ve toplam $\sum_{I=1}^{N-1}(N-I) = (N-1)+(N-2)+\cdots+1 = \frac{N(N-1)}{2}$ olur. N = 8 için bu 28'dir ve `9 8 7 6 4 3 2 1` dizisinde python ile de 28 sayıldı. Mertebe yine N^2'dir.
>
> **Hocanın vurgusu:** Algoritma karmaşıklığı ifade edilirken her zaman en yüksek üslü terim üzerinden ifade edilir. Örneğin N^3 olsaydı N^2'yi de atacaktık.
>
> **Ek bilgi (derste anlatılmadı):** En yüksek üslü terimle ifade etme fikri, kaynaklarda genellikle "büyük O" gösterimiyle $O(N^2)$ biçiminde yazılır. Bu gösterimle ilerleyen derslerde büyük olasılıkla karşılaşırsınız.

3.2 En İyi Durum (Best Case)

Sonra dizinin zaten sıralı verildiği `1 2 3 4 6 7 8 9` durumu soruldu.

Soru: Dizi sıralı verilirse en fazla kaç karşılaştırma yapılır? **Cevap:** Öğrenci, algoritmanın yine bütün ihtimalleri kontrol edeceğini, sadece değişiklik yapmayacağını, bu yüzden sonucun N^2 olacağını söyledi. Hoca onayladı: Bilgisayar dizinin sıralı olduğunu bilmediği için karşılaştırma yapmaya mecburdur. Tasarladığımız şekliyle algoritma en iyi durumda da N^2'lik karşılaştırma gerektirir, yalnızca yer değiştirme yapılmaz.

Hoca ekranda sıralı diziye yanlışlıkla "worst case" yazdığını fark edip best case olarak düzeltti.

3.3 En Zayıf Nokta: Sürekli Yer Değiştirme

Hocaya göre bu algoritmanın en zayıf noktası sürekli yer değiştiriyor olmasıdır. En kötü durum senaryosunda her turda çok sayıda yer değiştirme yapılır. Bu pek kabul edilebilir bir durum değildir. Çünkü her yer değiştirmede belleğe (memory) erişilir, bir eleman başka bir yere taşınır, onun yerine başka bir eleman yazılır, öteki eleman alınıp diğerinin yerine konur. Bellek erişimi açısından çok maliyetli bir algoritmadır ve bu anlamda en kötü durumlar için sevilmez.

> **Ders sonrası not:** Hoca "her turda N defa yer değiştiriyoruz" dedi; bu büyük olasılıkla mertebe için söylenmiştir. Doğrusu, I. turda en çok N−I yer değiştirme olur. `9 8 7 6 4 3 2 1` dizisinde python ile toplam 28 yer değiştirme sayıldı.

4. İyileştirilmiş Bubble Sort

4.1 Fikir: Değişiklik Olmadıysa Dizi Sıralıdır

Hoca, en iyi durum için algoritmayı iyileştirme şansımız olup olmadığını sordu. Bir öğrenci, iç döngü bittiğinde o turda dizide hiçbir değişiklik olmamışsa "break" türünden bir şey yapılabileceğini önerdi. Hoca fikrin geri kalanını doğru buldu, ama break kısmının unutulmasını istedi. Doğru tasarım, dış döngüde for yerine while, içte for kullanmaktır.

Sıralı dizide dış döngünün ilk turunda 1-2, 2-3, 3-4, 4-6, 6-7, 7-8 ve 8-9 çiftlerine bakılır. Hiçbiri değişmez. Hiçbiri değişmediyse dizi sıralı demektir.

Hoca sonra dizinin `2 1 3 4 6 7 8 9` verilmiş olabileceğini söyledi ve bu durumda ne olacağını sordu. Öğrencinin cevabı şuydu: İç döngüye bir kez girilir, ikinci kez de girilir, ama dizi değişmediği için üçüncü kez girilmez. Adım adım:

- Birinci turda 1 ile 2 yer değiştirir: `1 2 3 4 6 7 8 9`. Bu turda 7 karşılaştırma yapılır (ara adım eklendi).
- İkinci turda hiçbir yer değişmez. Bu turda 6 karşılaştırma yapılır (ara adım eklendi).
- Değişiklik olmadığı için dış döngü biter.

Hoca bunu, sıralıya yakın verilen dizileri daha az adımla çözmek olarak özetledi.

4.2 Değişken: flag

Dış döngüyü durdurmak için iç döngüde değişiklik olup olmadığını taşıyan bir değişken gerekir. Bir öğrenci de bunu önerdi. Hoca değişkene `flag` adını verdi.

1. İlk while kontrolünden geçilsin diye başta `flag = 1` yapılır. `I = 1` yapılır (ara adım eklendi).
2. `flag = 1` olduğu sürece dış while döner.
3. İç döngüye girmeden `flag = 0` yapılır. Sanki hiç değişiklik olmayacakmış gibi başlanır.
4. `J` değişkeni 1'den N−I'ya kadar döner.
5. `Dizi[J] > Dizi[J+1]` ise yer değiştirilir ve bu if'in true kolunda `flag = 1` yapılır.
6. İç döngü bitince `I` bir artırılır (ara adım eklendi) ve 2. adıma dönülür.
7. `flag` hâlâ 0 ise while'dan çıkılır ve dizi sıralıdır.

İç döngüden çıktıktan sonra ayrıca bir if koymaya gerek yoktur. Doğrudan while, değişkenin set edilip edilmediğine bakar.

Soru: Değişkeni tam olarak nerede tanımlayacağız? **Cevap:** While'a girmesi için en başta 1 olarak tanımlanır. Her turda iç döngüden önce 0 yapılır. True koluna girilirse 1 olur. Girilmezse değişken 0 olarak çıkar ve while bunu kontrol eder.

Soru: İç döngüde bir değişiklik olup `flag` 1 yapıldıktan sonra sonraki karşılaştırmalar false olduğunda değer 1 olarak mı kalır? **Cevap:** Öğrenci soruyu sormaktan vazgeçti, ama hoca yine özetledi: Değişken true kolunda set edilir. Döngü bittiğinde değişiklik olduysa yola devam edilir. İç döngüye yeniden girerken değer sıfırlanır, ki yeni turda gerçekten true koluna girilip girilmeyeceği anlaşılsın.

> **Hocanın vurgusu:** Değişkenin döngüde bir yer değişikliği olduğunu anlamasını istiyorsak true kolunda set etmemiz gerekir, ama döngüye girmeden de unset etmemiz gerekir.

Hoca bu versiyonu uzun uzun sözlü anlatmasının sebebini de söyledi. Çözümü yazsaydı üzerine kafa yorulmayacaktı. Akış diyagramının öğrenciler tarafından çizilmesini istedi, gerekirse birlikte çizileceğini belirtti. Yukarıdaki adımlar, tahtaya yazılan parçalar ve sözlü anlatımdan kurulmuştur.

4.3 İyileştirmenin Karmaşıklığa Etkisi

Soru: Bu iyileştirmeyle best case karmaşıklığı ne olur? **Cevap:** Bir öğrenci "yine N^2" dedi; hoca yanlış olduğunu söyledi. Bir başka öğrenci doğru cevabı açıkladı: Dizi sıralı olduğu için ilk döngüye girilir, N kez dönülür, değişken değişmediği için sıfır olarak geri dönülür. Sonuç N olur.

Hoca bunu sıralı dizi üzerinde tekrar etti: Birin ikiden, ikinin üçten, üçün dörtten, dördün altıdan, altının yediden, yedinin sekizden, sekizin dokuzdan büyük olup olmadığına baktı. Hiçbiri büyük değil, if'in true koluna girilmedi, while'a dönüldüğünde döngüden çıkıldı ve ilk turda iş bitti. En kötü durumda ise yapabileceğiniz bir şey yoktur. Sonuç olarak en kötü durum yine N^2, en iyi durum N'dir.

> **Ders sonrası not:** Hoca "N defa" dedi; bu büyük olasılıkla mertebe için söylenmiştir. Saydığı karşılaştırmalar 7 tanedir, yani tam sayı N−1'dir. Python ile sıralı 8 elemanlı dizide 7 karşılaştırma, `2 1 3 4 6 7 8 9` dizisinde 13 karşılaştırma ve 2 tur sayıldı. İyileştirme olmadan bu iki dizide de 28 karşılaştırma yapılır.

5. Selection Sort'un Mantığı

Bubble sort'un dezavantajının çok fazla bellek erişimi ve yer değiştirme olduğu söylendikten sonra selection sort'a geçildi. Bir öğrenci kendi araştırmasından şunu aktardı: Birinci elemanı seçiyor, hepsini kontrol ediyor, küçüğü en başa yerleştiriyor, sonra ikinciyi seçiyor. Hoca bu basit mantığı onayladı.

İlk eleman alınır, geri kalan elemanlarla karşılaştırılır, daha küçük olanla yer değiştirilir. Birinci turun sonunda başa en küçük eleman gelir. İkinci tura bu yüzden bir sonraki elemandan başlanır. Her turda en küçük elemanı bulup yerine yerleştirmenin peşindeyiz.

5.1 Örnek

Hoca bu örneği ilk versiyonla, yani daha küçük eleman görüldüğü anda yer değiştirerek yaptı. Dizinin sonuna 0 eklendi.

Başlangıç dizisi `1 3 5 4 2 6 8 7 0` olarak alındığında:

- Birinci tur: 1'den küçük tek eleman 0'dır. Yer değiştirir: `0 3 5 4 2 6 8 7 1`. Başa en küçük eleman gelmiştir.
- İkinci tur, 3 ile başlar:
  - 3 ile 5, 3 ile 4 karşılaştırılır. 3 ile 2 karşılaştırılınca 2 küçük olduğu için yer değişir: `0 2 5 4 3 6 8 7 1`.
  - 6, 8 ve 7, 2'den küçük değildir.
  - 1, 2'den küçüktür: `0 1 5 4 3 6 8 7 2`. En küçük ilk iki eleman bulunmuştur.
- Üçüncü tur, 5 ile başlar:
  - 4, 5'ten küçüktür: `0 1 4 5 3 6 8 7 2`.
  - 3, 4'ten küçüktür: `0 1 3 5 4 6 8 7 2`.
  - En sonda 2, 3'ten küçüktür: `0 1 2 5 4 6 8 7 3`. En küçük üç eleman bulunmuştur.

5.2 Sorular

Soru: Sürekli yer değiştirmek yerine bütün elemanları birer sağa kaydırıp başa atsak daha kısa sürede bitmez mi? **Cevap:** Hoca bu şekilde doğru sonuca ulaşılamayacağını söyledi.

Soru: Araştırdığımda minimum bulunduğunda hemen değiştirmiyor, yerini saklıyor, en sonda bir değişiklik yapıyordu. Hangisi doğru? **Cevap:** Kafanızın karışmasına gerek yok. Selection sort'un ilk versiyonu her karşılaştırmada kendinden küçük eleman görünce yer değiştirir. Bahsedilen versiyon ise tur bitmeden yer değiştirmez, tur bitince en küçük elemanla elimizdeki elemanın yerini değiştirir. Efektif çözüm de budur.

> **Hocanın vurgusu:** Sort mantığını oturtursanız bundan sonraki süreçte çok rahat edersiniz. Bu soruyu mutlaka çözmeye çalışın, çok önemli.

6. Selection Sort'un İlkel Versiyonu

6.1 Akış

Aradan sonra algoritma öğrencilerle birlikte çizildi. Bir öğrencinin önerisiyle dış döngünün sonu N−1, iç döngü ise I+1'den N'e kadar alındı.

1. Başla. N ve diziyi al.
2. `I` değişkenini 1'den N−1'e kadar birer birer artır.
3. Her `I` için `J` değişkenini I+1'den N'e kadar birer birer artır.
4. `Dizi[I] > Dizi[J]` ise yer değiştir: `tmp = Dizi[J]`, `Dizi[J] = Dizi[I]`, `Dizi[I] = tmp`.
5. Döngüler bitince dur. Hoca bu örnekte diziyi ekrana yazdırmadı, ama yazdırılabileceğini söyledi.

Akış çizilirken ekrandaki yer değiştirmede tmp yazılması unutulmuştu. Bir öğrenci uyardı, hoca düzeltti ve bu kısmın önemli olduğunu söyledi. Bu işleme **swap** denir. İki elemanın yerini değiştirmek için ekstra üçüncü bir alan kullanılır.

Soru: Swap kısmında I ile J'nin yeri değişse, yani önce `tmp = Dizi[I]`, sonra `Dizi[I] = Dizi[J]`, sonra `Dizi[J] = tmp` yazsak fark eder mi? **Cevap:** Hiçbir şey değişmez. Hangi taraftan baş-

ladığınızın önemi yoktur. Birini tmp alanına yazarsınız, aldığınızın yerine diğerini yazarsınız, sonra tmp'den son değişikliği yaparsınız.

6.2 Örnek: 5 1 3 2 4 8 0

Elemanlar 1'den 7'ye numaralanır, N = 7'dir.

Birinci tur, I = 1:

- J = 2: `Dizi[1] = 5`, `Dizi[2] = 1`. 1, 5'ten küçüktür, yer değişir: `1 5 3 2 4 8 0`.
- J = 3, 4, 5, 6: 3, 2, 4, 8 sayıları 1'den küçük değildir.
- J = 7: `Dizi[1] = 1`, `Dizi[7] = 0`. Yer değişir: `0 5 3 2 4 8 1`.

Birinci tur tamamlanmıştır, 0'ın yeri artık sabittir.

İkinci tur, I = 2, J 3'ten başlar:

- J = 3: 3, 5'ten küçüktür: `0 3 5 2 4 8 1`.
- J = 4: 2, 3'ten küçüktür: `0 2 5 3 4 8 1`.
- J = 5 ve 6: 4 ve 8, 2'den küçük değildir.
- J = 7: 1, 2'den küçüktür: `0 1 5 3 4 8 2`.

İkinci turun sonunda en küçük iki eleman bellidir. Hoca kalan turları yapmadı, sadece N−1. turun sonunda dizinin `0 1 2 3 4 5 8` olacağını söyledi. Python ile aynı sonuç elde edildi.

6.3 Sıkıntı Nedir?

Hoca bu halin sıkıntısını sordu. Bir öğrenci şunu söyledi: Normalde en küçüğü bulmaya çalışıyoruz, ama burada bir küçüğü bulup değiştiriyor, sonra yine bir küçüğü bulup değiştiriyoruz, bu da fazla işlem alıyor. Onun yerine bütün turda en küçüğün yeri bulunabilir ve değişiklik iç döngünün dışında, en sonda yapılabilir.

7. İyileştirilmiş Selection Sort

7.1 Akış

Hoca, en küçüğün yerini iç döngü boyunca bulup yer değiştirmeyi iç döngüden çıktıktan sonra yapmayı istedi. Tasarımı bir öğrenci söyledi:

1. Başla. N ve diziyi al.
2. `I` değişkenini 1'den N−1'e kadar birer birer artır.
3. Birinci for'dan çıkıp ikinci for'a girmeden `min = I` yap.
4. `J` değişkenini I+1'den N'e kadar birer birer artır.
5. `Dizi[J] < Dizi[min]` ise `min = J` yap.
6. İç döngü bitince yer değiştir: `tmp = Dizi[I]`, `Dizi[I] = Dizi[min]`, `Dizi[min] = tmp`.
7. Döngüler bitince dur.

Hoca buradaki koşulu özellikle önemli buldu: `Dizi[J] < Dizi[min]`. Dizi boyunca elimizdeki en küçük eleman tutulur ve ondan daha küçük bir eleman olup olmadığına bakılır.

İç döngüde daha küçük bir eleman varsa yalnızca onun **yeri** saklanır, değeri değil. Değerini de saklayarak yapılan çözümler vardır ve yanlış değildir. Ama çözüm daha iyi olsun diye yalnız yer saklanır.

7.2 Örnek: 5 1 3 2 4 8 0

Birinci tur, I = 1 ve `min = 1`, yani elimizdeki en küçük eleman 5'tir:

- 1, 5'ten küçüktür: `min = 2`.
- 3, 2, 4, 8 sayıları 1'den küçük değildir.
- 0, 1'den küçüktür: `min = 7`.

Bu turda bulunan en küçük eleman 7. gözdedir. Önce 1. gözdeki `tmp`'ye alınır, 7. gözdeki 1. göze atanır, sonra `tmp`'deki 7. göze konur. Sonuç `0 1 3 2 4 8 5` olur.

Bu turda dış döngünün bir turu için tek bir yer değiştirme yapılmıştır. Önceki çözümde ise en kötü durum senaryosunda bir turda N−1 değişiklik yapmak gerekir. Hoca bunu oldukça kötü bir tasarım olarak niteledi.

> **Ders sonrası not:** Hoca N−1 değişikliği "en büyük eleman baştaysa" diye anlattı. Doğrusu, bunun için ilk elemanın en büyük olması yetmez, geri kalan elemanların da azalan sırada gelmesi gerekir. Python ile `8 5 4 3 2 1 0` dizisinde ilkel versiyonun birinci turunda 6 değişiklik sayıldı. Hoca bu sayımı büyük olasılıkla azalan diziyi düşünerek yapmıştır. `5 1 3 2 4 8 0` dizisinin tamamında ilkel versiyon 11, iyileştirilmiş versiyon 6 yer değiştirme yapar.

İkinci turda I = 2 olur. `min` de 2'dir, yani elimizdeki değer 1'dir. İç döngü I+1'den, yani 3'ten başlar. 3, 2, 4, 8 ve 5 sayılarının hiçbiri 1'den küçük değildir. İç döngüden çıkılır.

7.3 Kendi Üzerine Yazma

Daha küçük eleman yoksa ne olur? Eleman `tmp`'ye alınır, kendi üzerine yazılır, `tmp`'deki bir daha aynı yere alınır. Bir öğrenci, kendi kendine yer değiştirmeyi önlemek için başa `min ≠ I` kontrolü eklediğini söyledi. Bir başka öğrenci de ilk eleman en küçükse değiştirmeye gerek olmadığı için if yazmayı önermişti.

> **Hocanın vurgusu:** Hayır, böyle bir if konmaz. Hangi elemanın en küçük olduğunu önceden bilemezsiniz. Evet, fazla bir işlem yapılıyormuş gibi görünür, ama her turda kontrol etmek yerine bu kendi üzerine yazmayı kabul ediyorsunuz. Daha küçüğü yoksa ekstra kontrol koymamanız gerekir.

7.4 Tasarruf ve Karmaşıklık

Hoca kazancı şöyle özetledi: Her seferinde yer değiştirmiyoruz, en küçüğü her seferinde almak yerine turun sonunda olması gereken yere alıyoruz.

> **Ders sonrası not:** Hoca bu özette "ikili karşılaştırma yapmıyoruz" ve "ikili karşılaştırma en maliyetli olanı" dedi. Doğrusu, selection sort da karşılaştırma yapar ve karşılaştırma sayısı azalmaz. Hocanın dersin önceki kısmında anlattığı maliyet bellek erişimi ve yer değiştirmedir, burada da büyük olasılıkla bubble sort'taki yan yana karşılaştırıp hemen yer değiştirmeyi kastetmiştir. Kazanç yer değiştirme sayısındadır.

Soru: Selection sort'ta en kötü durumda karşılaştırma sayısı nedir? **Cevap:** Bir öğrenci N^2 dedi. Kötü durum senaryosu olarak büyükten küçüğe giden diziyi gösterdi. Hoca bunu ekrandaki elemanlarla `8 5 4 3 2 1 0` olarak ele aldı: Elimize 8 alınır, 5 ile karşılaştırılır, daha küçüktür. 4 ile karşılaştırılır, daha küçüktür. En sona kadar gidilir. Bir tur sonunda dizi `0 5 4 3 2 1 8` olur, sırada 5 vardır. Bu şekilde N defa ilerlenir. N çarpı N'den N^2'lik karşılaştırma öngörülür.

Soru: En iyi durumda, dizi `1 2 3 4 5 8` gibi sıralı verilirse ne olur? **Cevap:** Öğrenci, kontrolün her türlü yapılacağını, değişiklik yapılmayacağını söyledi. Hoca onayladı. Hatta kendi üzerine yazma yine yapılır, ama if'in true koluna girilmez. Selection sort'un en iyi durum senaryosunda da N^2'lik kontrol gerekir.

> **Ders sonrası not:** Hocanın "N çarpı N" ifadesi büyük olasılıkla mertebe içindir. Bubble sort'taki gibi tam karşılaştırma sayısı $\frac{N(N-1)}{2}$'dir. Python ile `8 5 4 3 2 1 0` ve `0 1 2 3 4 5 8` dizilerinde 21'er karşılaştırma ve 6'şar yer değiştirme sayıldı.

7.5 Bir İyileştirme Önerisi

Soru: İç döngüden hemen sonra `min` değeri I'ya eşit oluyorsa minimumlar hep soldadır, döngü bitirilebilir mi? **Cevap:** Hoca düşünebilmek için sayısal bir örnek istedi. Öğrenci örnek ararken böyle bir senaryonun olamayacağını düşündüğünü söyledi ve önerisini geri çekti. Hoca önemli olmadığını söyledi ve bu soruyu öğrencilerin kendilerine bıraktı.

Ders sonrası not: Bu ölçüt büyük olasılıkla işe yaramaz, çünkü dersin kendi örneği bunu gösterir. `0 1 3 2 4 8 5` dizisinin ikinci turunda `min = I = 2` kalır, ama sağ taraftaki `3 2` çifti hâlâ sırasızdır. Burada durulsaydı dizi sıralanmamış olurdu.

Ek bilgi (derste anlatılmadı): Derste C kodu yazılmadı. Aşağıdaki kod, iyileştirilmiş bubble sort ve iyileştirilmiş selection sort akışlarının C'ye çevrilmiş halidir ve tahtadaki yazımla büyük olasılıkla birebir aynı değildir. C'de indisler 0'dan başladığı için bubble sort'ta öğrencinin söylediği `N-i-1` sınırı kullanılır. Kod `gcc -Wall` ile uyarısız derlendi ve dersteki dizileri doğru sıraladı.

```c
void bubble_iyilestirilmis(int dizi[], int N)
{
    int i = 0, j, tmp, flag = 1;
    while (flag == 1) {
        flag = 0;
        for (j = 0; j < N - i - 1; j++) {
            if (dizi[j] > dizi[j + 1]) {
                tmp = dizi[j];
                dizi[j] = dizi[j + 1];
                dizi[j + 1] = tmp;
                flag = 1;
            }
        }
        i++;
    }
}

void selection_iyilestirilmis(int dizi[], int N)
{
    int i, j, min, tmp;
    for (i = 0; i < N - 1; i++) {
        min = i;
        for (j = i + 1; j < N; j++) {
            if (dizi[j] < dizi[min])
                min = j;
        }
        tmp = dizi[i];
        dizi[i] = dizi[min];
        dizi[min] = tmp;
    }
}
```

8. Insertion Sort'un Mantığı

8.1 Kağıt Dizme Benzetmesi

Son olarak insertion sort'un mantığı tanıtıldı. Hoca önce bilen olup olmadığını sordu. Bir öğrenci, minimumu bulup en sağına ekleyip hepsini sola kaydırmayı "biraz sallayarak" önerdi. Hoca bunun sallama olduğunu, ama üzerine gidilirse bu fikirden bir şey çıkabileceğini söyledi.

Sonra kağıt oyununu örnek verdi. Kağıtlar elinize karışık gelir ve siz onları sıralarsınız. Mesela elinizde 5 ve 8 var, burada da 7 var. Ne yaparsınız? 7'yi araya sokarsınız. Insertion sort da aynen bu mantığa dayanır: elemanları olmaları gereken pozisyona yerleştirmek.

8.2 Örnek

Ders sonrası not: Altyazıda başlangıç dizisi `5 2 3 1 4 7 8` olarak geçiyor; hocanın ara dizileri bununla uyuşmuyor. Dizi büyük olasılıkla `5 3 2 1 4 7 8` olmalıdır, çünkü söylenen ara diziler python ile bu başlangıçtan elde edildi. Aşağıdaki adımlar bu diziyle yazılmıştır.

Adımlar şöyledir:

- 3 başa alınır: `3 5 2 1 4 7 8`. İlk iki eleman kendi arasında sıralıdır.
- 2 başa alınır: `2 3 5 1 4 7 8` (ara adım eklendi).
- 1 başa alınır: `1 2 3 5 4 7 8`.
- 4, 3 ile 5'in arasına alınır: `1 2 3 4 5 7 8`. Dizi sıralanmıştır.

Bir öğrenci bu mantığı şöyle anlattı: Önce 5 ile 3 kendi arasında, sonra 5, 3 ve 2 kendi arasında, sonra 5, 3, 2 ve 1 kendi arasında sıralanır. Hoca sonunda yapılması gereken şeyin 4 ile 5'in yerini değiştirmek olduğunu söyledi. Yapmak istediğimiz şey, elemanları olmaları gereken sıraya getirmektir.

> **Hocanın vurgusu:** Insertion sort'u çözerken nasıl çözüldüğüne bir yerden bakmayın, yoksa hiçbir esprisi kalmaz. Önemli olan ilgili elemanı olması gereken pozisyona getirmektir.

Hoca insertion sort'un algoritmasını derste yazmadı. Bunu ve iyileştirilmiş bubble sort'u öğrencilerin kendi başına çözmesini istedi. Çözülemezse derste birlikte bakılacağını söyledi.

8.3 Kağıtların Rengi

Soru: Kağıtları sıralarken sinek, maça gibi renklerine göre de ayırıyor muyuz? **Cevap:** Hayır. Burada kağıt yok, yalnız sayı var. Hoca kağıtlarda sayısal sıralamayı sordu. Bize sinek ve maça dağıtılıp bunların sıralanması istenseydi soru farklı olurdu. Öğrenci kağıtları hem renklerine göre ayırıp hem de kendi içinde sıralamanın güzel bir problem olacağını söyledi. Hoca da bunun güzel bir soru olduğunu ve gerçek hayat problemi olduğunu kabul etti. Bilgisayarda oynanan kağıt oyunlarında yapay zekanın da bu mantıkla işlem yaptığı konuşuldu.

Dersin Toparlaması

Bu derste üç sıralama algoritması konuşuldu. Bubble sort, yan yana elemanları karşılaştırıp her turda en büyüğü sona iter. En kötü durumda N^2 karşılaştırma ve çok sayıda yer değiştirme yapar. `flag` değişkeniyle ve dışta while kullanılarak iyileştirildiğinde, en iyi durum N^2'den N'e iner, en kötü durum değişmez. Selection sort, her turda kalan elemanların en küçüğünü bulur. İyileştirilmiş versiyonu en küçüğün yalnızca yerini saklar ve tur sonunda tek bir swap yapar. En iyi ve en kötü durumda N^2 karşılaştırma gerekir, kazanç yer değiştirme sayısındadır.

Insertion sort'un yalnızca mantığı anlatıldı: Her eleman, kağıt dizer gibi olması gereken pozisyona yerleştirilir. Algoritmayı kendi başınıza kurmaya çalışmanız isteniyor. İyileştirilmiş bubble sort ve insertion sort'un çözümüne bir sonraki derste bakılacak, sonra yeni konuya geçilecektir.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.